



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

策略模式（Strategy）

一、意图（Intent）

定义一系列算法，把它们一个个封装起来，并且使它们可以相互替换。

在嵌入式 C 里，这句话更应该理解成：

上层面对的是同一类能力接口，至于底下具体采用哪套实现策略，由运行时或编译期装配决定。

这个模式控制的是：

- 同一功能的多种实现选择
- 算法 / 驱动后端的替换边界
- 上层对下层策略的依赖方式
- 选择策略的时机（编译期 / 初始化期 / 运行期）

一句话记住它：

上层只说“我要做这件事”，至于用哪种做法，由策略层决定。

二、动机（Motivation）

嵌入式系统里，“同一个动作有多种实现”是很常见的：

- 不同芯片的 UART / SPI / I2C 驱动实现不同
- 存储后端可能是 NOR Flash、NAND、EEPROM、NVS

- 过滤算法可能有轻量版、精确版、省电版
- 日志输出可能走 UART、RTT、Flash、网络
- 通信重试策略可能有快速重试、指数退避、静默失败

最直接的坏味道通常是：

- 上层充满 `if (chip_xxx)`
- 上层自己判断是否使用某种算法
- 每增加一个平台就多一层分支

问题会越来越明显：

1. 上层被具体实现污染

业务层本来只想“发一个字节”“存一个配置”“输出一条日志”，结果却必须知道底层是哪家芯片、哪种介质、哪条总线。

2. 替换成本高

新增一种实现，就要改上层调用路径。

3. 无法形成统一能力接口

系统明明做的是同一类事情，却没有稳定的抽象边界。

所以策略模式在嵌入式里真正解决的是：

把“要做什么”和“具体怎么做”隔离开，让实现策略可替换。

三、适用性（Applicability）

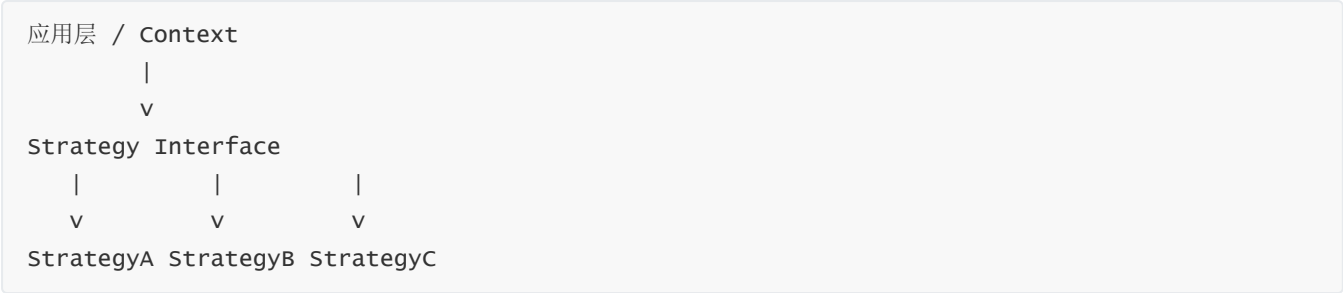
适合使用策略模式的条件包括：

- 同一个能力存在多种实现方案
- 上层只关心统一行为，不应关心具体实现细节
- 不同产品 / 板级 / 芯片需要切换实现
- 希望通过函数表或接口表完成解耦
- 算法选择或驱动选择可能随配置变化

常见场景：

- 驱动后端抽象
 - 存储后端抽象
 - 编解码算法切换
 - 过滤 / 控制算法切换
 - 日志 / 网络 / 文件系统后端切换
-

四、结构（Structure）



在 C 里通常长这样：

```
ctx
|
+-- 指向一张 ops 表
    |
    +-- init()
    +-- read()
    +-- write()
    +-- control()
```

所以策略模式在嵌入式 C 里的典型落地，不是类继承，而是：

- 一份统一接口定义
- 多个实现函数集
- 一个函数指针表 / API 表
- 上层持有当前策略引用

五、参与者（Participants）

1. Context（上下文）

负责：

- 持有当前策略
- 在合适的时候调用策略接口
- 保存与策略无关但与业务有关的上下文数据

2. Strategy（策略接口）

定义统一能力边界。

在 C 中通常体现为：

- 一组函数原型
- 一个 API 结构体
- 一套约定好的返回值语义

3. ConcreteStrategy（具体策略）

实现某一套具体行为。

比如：

- 不同芯片驱动后端
- 不同存储介质后端
- 不同滤波算法
- 不同重试策略

六、协作（Collaborations）

整体流程很简单：

1. 系统初始化时选定某个策略
2. Context 保存该策略引用
3. 上层调用统一接口
4. 实际行为落到对应策略实现

要点在于：

- 调用方不再写平台分支
- 切换实现时尽量不改业务层
- 策略可以按产品线装配

七、效果（Consequences）

收益

1. 统一上层调用面

应用层看到的是稳定接口，而不是五花八门的底层实现。

2. 更利于产品线扩展

同一套业务代码可以挂接不同策略实现。

3. 替换实现更自然

新增平台或新增算法时，影响主要落在策略实现层。

4. 适合函数表风格的 C 工程

这种模式和嵌入式里广泛使用的 `ops / api / driver table` 非常契合。

代价

1. 多一层间接调用

会带来一点点指针跳转和理解成本。

2. 接口设计必须稳定

策略接口一旦设计不好，后续所有实现都会受影响。

3. 简单场景可能显得偏重

如果永远只有一种实现，引入策略层可能没有必要。

八、实现要点 (Implementation)

1. 接口只放“共同能力”

不要把某个特定平台的私有能力硬塞进公共接口。

2. 明确上下文所有权

策略实现是否持有设备私有数据、由谁初始化、生命周期归谁，都要说清。

3. 区分编译期选择和运行期选择

嵌入式里很多“可替换”其实是在初始化时就定死了，不一定真的支持热切换。

4. 返回值和错误语义要统一

否则上层虽然接口统一了，错误处理还是散的。

5. 控制私有扩展口

若必须支持某些后端私有扩展，最好通过单独扩展接口而不是污染主接口。

九、典型应用

1. 同一功能的多后端存储

例如设置保存可以落到不同存储介质；上层只关心“保存配置”，不关心底下到底是 NVS、FCB 还是文件系统。Zephyr settings 也明确提供多种后端实现。[□cite□turn609390search3□](#)

2. 多种算法切换

例如电机控制里切换不同控制律、滤波模块切换不同滤波器、通信模块切换不同重传策略。

总结

策略模式在嵌入式 C 里，最常见的落地就是“统一接口 + 多套 ops 表实现”，让业务层不再被具体驱动或算法绑死。